

PYTHON - SE - BACKEND

SOFTWARE ENGINEERING – MODULE 4

Database Management – SQL & PL/SQL

SESSION / ASSIGNMENT – 6

TASKS :

1. Create a SQL query using GROUP BY to show the total number of orders placed by each user in an 'orders' table, displaying user_id and order_count.

→ CODE:

```
SELECT user_id, COUNT(*) AS order_count
FROM orders
GROUP BY user_id;
```

OUTPUT:

✓ Showing rows 0 - 2 (3 total, Query took 0.0011 seconds.)

```
SELECT user_id, COUNT(*) AS order_count FROM orders GROUP BY user_id;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: Filter rows:

Extra options

user_id	order_count
1	2
2	3
3	1

2. Write a SQL query to display the names of restaurants from a 'restaurants' table that have received an average rating above 4.0, using GROUP BY and HAVING.

→ CODE:

```
SELECT name, AVG(rating) AS average_rating
FROM restaurants
GROUP BY name
HAVING AVG(rating) > 4.0;
```

OUTPUT:

✓ Showing rows 0 - 4 (5 total, Query took 0.0011 seconds.)

```
SELECT name, AVG(rating) AS average_rating FROM restaurants GROUP BY name HAVING AVG(rating) > 4.0;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 Filter rows:

Extra options

			name	average_rating	
☐	Edit	Copy	Delete	The Spice House	4.50000
☐	Edit	Copy	Delete	Sushi World	4.20000
☐	Edit	Copy	Delete	La Bella Italia	4.70000
☐	Edit	Copy	Delete	Taco Fiesta	4.10000
☐	Edit	Copy	Delete	Dragon Palace	4.40000

3. Suppose you have a 'payments' table with columns user_id, amount, and payment_date. Write a subquery to find user_ids who have made a single payment above ₹2000 in any transaction.

➔ CODE:

```
SELECT DISTINCT user_id
FROM payments
WHERE user_id IN (
SELECT user_id
FROM payments
WHERE amount > 2000
);
```

OUTPUT:

✓ Showing rows 0 - 1 (2 total, Query took 0.0026 seconds.)

SELECT DISTINCT user_id FROM payments WHERE user_id IN (SELECT user_id FROM payments WHERE amount > 2000);

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Extra options

user_id
1
3

4. Using a nested subquery, display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies.

Hint: Use a subquery to calculate the overall average rating, then compare each movie's average rating against it in the main query.

➔ **CODE:**

```
SELECT name, AVG(rating) AS average_rating
FROM movies
GROUP BY name
HAVING AVG(rating) > (
SELECT AVG(rating)
FROM movies
);
```

OUTPUT:

✓ Showing rows 0 - 1 (2 total, Query took 0.0032 seconds.)

SELECT name, AVG(rating) AS average_rating FROM movies GROUP BY name HAVING AVG(rating) > (SELECT AVG(rating) FROM movies);

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

	name	average_rating
☐ Edit Copy Delete	Inception	4.70000
☐ Edit Copy Delete	Interstellar	4.80000